

En el inicio del ciclo 3 se realizó un análisis del estado actual del proyecto y de los nuevos requerimientos del documento. Se identificó que la base de simulación construida en los ciclos anteriores ya estaba lista para reutilizarse, pero que aún no existía una clase separada para resolver el problema de la maratón. A partir de esto se definieron mini-ciclos enfocados en crear TowerContest, implementar el algoritmo de solución, conectar esa solución con la simulación y reforzar las pruebas requeridas para la entrega.

Mini-ciclo 1: TowerContest

Objetivo: Crear la clase TowerContest con la estructura mínima:

- solve(int n, int h)
- simulate(int n, int h)

Qué se hace:

- crear la clase,
- definir claramente qué devuelve solve,
- decidir el formato exacto del string de salida,
- preparar la nueva clase de pruebas TowerContestTest.

Justificación: Antes de programar el algoritmo, necesitamos dejar clara la interfaz del ciclo 3 y separar bien solver vs simulador. Este mini-ciclo reduce el riesgo de mezclar responsabilidades.

Clases esperadas

- **Añadida:** TowerContest
- **Añadida:** TowerContestTest

Mini-ciclo 2: solve(n, h)

Objetivo: Implementar el algoritmo que determine si existe una secuencia de las n tazas cuya altura total sea exactamente h, y devolver una respuesta válida o imposible.

- analizar la fórmula de altura,
- construir una estrategia correcta,
- devolver el orden como string,
- probar casos pequeños y casos imposibles.

Justificación: Este es el núcleo del ciclo 3, porque el documento dice que en esta entrega se debe resolver el problema de la maratón.

Clases esperadas

- **Modificada:** TowerContest
- **Modificada:** TowerContestTest

Mini-ciclo 3: conectar solve con simulate

Objetivo: Implementar `simulate(n, h)` para que use la solución encontrada y la represente con Tower.

- llamar internamente a `solve(n, h)`,
- si hay solución, crear una Tower,
- insertar las tazas en el orden correcto,
- mostrar visualmente la torre,
- si no hay solución, mostrar mensaje o comportamiento definido.

Justificación: Con esto ya se cumple el requisito 15: **simular la solución, si es posible**.

Clases esperadas

- **Modificada:** TowerContest
- posiblemente una clase de demo o acceptance para la simulación

Mini-ciclo 4: pruebas fuertes y casos comunes

Objetivo: Agregar cobertura seria a TowerContestTest y construir TowerContestCTest.

- probar salidas correctas,
- probar imposibles,
- probar que la salida tenga exactamente n elementos,
- probar que la simulación no falle en casos válidos e inválidos.

Justificación: El documento pide explícitamente ambas clases de prueba para este ciclo.

Clases esperadas

- **Modificada:** TowerContestTest
- **Añadida:** TowerContestCTest

Mini-ciclo 5: cierre y validación de integración

Objetivo: Verificar que solver y simulador estén bien separados y documentados.

- revisar Javadoc,
- revisar nombres,
- limpiar helpers innecesarios,
- validar aceptación visual,
- dejar lista la retrospectiva.

Justificación: Este ciclo ya no es solo de extensión, también es de perfeccionamiento y calidad, como dice el documento.

Mini ciclo 1: TowerContest

En este mini-ciclo se creó la clase TowerContest, que será la encargada de resolver el problema de la maratón sin mezclar esa lógica con la clase Tower, ya que Tower debe usarse solo para simular. También se implementaron los primeros casos base de solve, cubriendo la altura mínima, la altura máxima, el caso más pequeño posible y las situaciones imposibles por rango. Además, se creó la clase TowerContestTest para comenzar a validar el comportamiento del nuevo solver desde etapas tempranas.

Clases añadidas: TowerContest, TowerContestTest

Clases modificadas: ninguna

Métodos añadidos:

- solve(int n, int h)
- simulate(int n, int h)
- isValidCupCount(int n)
- getMinHeight(int n)
- getMaxHeight(int n)
- buildDescendingSolution(int n)
- buildAscendingSolution(int n)

Mini ciclo 2: solve(n, h)

En este mini-ciclo se completó la lógica principal de solve para que ya no se limitara a los casos base, sino que también resolviera alturas intermedias válidas. La idea fue construir la solución de forma recursiva, separando claramente los casos donde conviene ubicar primero la taza más grande y los casos donde conviene dejar algunas tazas antes de ella. Además, se actualizaron las pruebas de TowerContestTest para validar soluciones reales y no solo respuestas exactas, comprobando que la secuencia encontrada use todas las tazas y produzca la altura solicitada al simularla.

Clases modificadas: TowerContest, TowerContestTest

Clases añadidas: ninguna

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

En este momento consideramos que el proyecto quedó en un estado parcial del ciclo 3, específicamente con los mini-ciclos 1 y 2 desarrollados y los demás solo planeados. Esto significa que ya existe una base funcional del solver en la clase TowerContest, y que alcanzamos a resolver casos base y varias alturas intermedias, pero todavía no se cerró completamente la parte de integración visual ni la validación final del ciclo.

Decimos que quedó parcial porque, aunque sí avanzamos en la parte algorítmica más importante, no logramos completar todo lo planeado para el ciclo. En otras palabras, el proyecto no está detenido por un error técnico puntual sino por tiempo, ya que sí se dejó una base organizada para retomar el trabajo después.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes?

Aquí deben poner sus tiempos reales. Una forma breve de escribirlo sería:

- Yeray: 7 horas
- Andrés: 7 horas

4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?

Consideramos que el mayor logro de este ciclo fue haber logrado separar correctamente el solver de la simulación. En los ciclos anteriores casi todo giraba alrededor de Tower, pero en este ciclo ya era necesario pensar en una clase distinta, TowerContest, que resolviera el problema sin depender de la parte gráfica.

También fue un logro importante haber alcanzado una primera solución funcional para alturas intermedias, porque eso nos confirmó que sí íbamos por una dirección válida en la resolución del problema y que no nos estábamos quedando solo en los casos fáciles.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue entender bien el problema de la maratón y cómo construir una solución correcta para la altura pedida. En este ciclo ya no bastaba con agregar métodos a un simulador; ahora tocaba pensar en términos más algorítmicos y encontrar una forma válida de ordenar las tazas para producir exactamente una altura h .

Lo más difícil no fue tanto programar la clase nueva, sino entender:

- cuáles alturas eran posibles,
- cuáles no,
- y cómo construir una solución sin repetir tazas y usando todas las disponibles.

Para resolverlo, lo que hicimos fue dividir el problema en partes más pequeñas. Primero trabajamos los casos base, después las fronteras de altura mínima y máxima, y solo luego pasamos a las alturas intermedias. También cambiamos el enfoque de las pruebas: en vez de exigir siempre una cadena exacta, validamos que la solución construida fuera consistente y que efectivamente generara la altura correcta al usar Tower.

6. ¿Qué hicieron bien como equipo? ¿A qué se comprometen a hacer para mejorar los resultados?

Como equipo creemos que hicimos bien la organización por mini-ciclos, porque eso evitó que intentáramos resolver todo de una sola vez. Además, cuando vimos que el ciclo 3 ya exigía una solución más matemática, no tratamos de forzar esa lógica dentro de Tower, sino que aceptamos que era mejor separar responsabilidades en su propia clase.

También fue positivo que no nos quedamos solo con la idea inicial de las pruebas, sino que entendimos que para un problema como este a veces no basta con verificar un string exacto sino que hay que comprobar si la solución realmente es válida.

Como compromiso para mejorar, creemos que en un siguiente momento deberíamos:

- dedicar más tiempo al análisis del problema antes de programar,
- validar antes los casos matemáticos especiales,
- y dejar más documentadas las decisiones que se vayan tomando durante cada mini-ciclo.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Para este ciclo usamos como base los documentos de trabajo del proyecto, pero para la parte técnica también nos apoyamos en fuentes oficiales de Java y en ejemplos de la comunidad. Las más útiles fueron las páginas oficiales de Oracle sobre polimorfismo, documentación Javadoc y las clases de la biblioteca estándar que usamos en la implementación, como ArrayList y StringJoiner. Además, consultamos un ejemplo en Stack Overflow para contrastar ideas de implementación y validar que nuestro enfoque no estuviera tan alejado de soluciones comunes (muy similar al ciclo pasado ya que nos ayudó bastante).

La referencia que más nos sirvió en este ciclo fue realmente el documento del proyecto, porque ahí estaba el contexto específico del problema; pero de las fuentes externas, la más útil fue la de Oracle sobre polimorfismo, ya que nos ayudó a mantener clara la separación entre responsabilidades de las clases, y la especificación de Javadoc nos sirvió para documentar mejor los métodos nuevos. Las referencias de ArrayList y StringJoiner también fueron útiles porque esas estructuras se usaron directamente en la construcción de la solución.

Referencias:

- Oracle. (s. f.). *Polymorphism*. The Java Tutorials. Recuperado el 15 de marzo de 2026, de [Oracle Java Tutorials - Polymorphism](#)
- Oracle. (s. f.). *StringJoiner (Java SE 12 & JDK 12)*. Recuperado el 15 de marzo de 2026, de [Oracle API - StringJoiner](#)
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *DOPO-I03-2026-01: Documento del ciclo 3 del proyecto*.
- Escuela Colombiana de Ingeniería Julio Garavito]. (2026). *J-stackingcups: Enunciado base del problema*
- Paulo Bu. (2014, 28 de marzo). *Mastering recursive programming* [Foro de discusión]. Stack Overflow.
<https://stackoverflow.com/questions/22713811/mastering-recursive-programming>
- ProCode Tv. (2016, 12 de octubre). *Programacion orientada a objetos en JAVA - POLIMORFISMO* [Video]. YouTube. <https://www.youtube.com/watch?v=5z-trhKdL-4>